

TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

REPORT ON LABORATORY WORK NO. 4

OPERATING SYSTEMS

The xv6 Shell from the Outside

Done by: st. gr. FAF-241, Lucian-Adrian Gavril

Checked by: univ. assist., Reițman Patricia

Chișinău, 2026

TABLE OF CONTENTS

INTRODUCTION 3
1 FIX SLEEP FOR THE CURRENT XV6 4
1.1 Which name does the tree use? 4
1.2 The updated sleep.c 4
1.3 Running sleep in xv6 5
2 BASH VERSUS THE XV6 SHELL 6
2.1 Comparison table 6
2.2 ls after cd d 9
2.3 Answers to the Observe questions 9
3 WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV 10
3.1 args.c and args_linux.c 10
3.2 Answers to the Observe questions 12
4 READ THE SHELL 14
4.1 How sh.c runs a line 15
4.2 Answers to the Observe questions 16
CONCLUSIONS 17
REFERENCES 18

INTRODUCTION

Purpose of the work. To use the xv6 shell (`user/sh.c`, 499 lines) side by side with Bash and find out by experiment what a shell must do and what Bash adds on top. To show with a small program, `args`, who expands `*` and who removes quotes. To bring the `sleep` program from Lab 2 up to date with the current xv6, where the system call `sleep()` has been renamed `pause()`.

Environment. Debian GNU/Linux 13, GNU Bash 5.2.37, GCC 14, `riscv64-linux-gnu` toolchain, QEMU 10 (`qemu-system-riscv64`). xv6 is the current upstream `mit-pdos/xv6-riscv` in `~/xv6-riscv` (commit `e93dabe`), to which `user/sleep.c` and `user/args.c` were added. Each QEMU session started from a freshly built `fs.img`. Every terminal figure is the real output of the commands shown in it.

1 FIX SLEEP FOR THE CURRENT XV6

1.1 Which name does the tree use?

`grep -n "pause\|sleep" user/user.h` prints a single line, `25:int pause(int);`, so this tree uses the new name and has no `sleep()` declaration at all (Figure 1.1). The matching stub is `entry("pause")` in `user/usys.pl`. A `sleep.c` that still calls `sleep()` therefore fails at link time with `undefined reference`: declaring a prototype in `user.h` is not enough, because a system call also needs the assembly stub that `usys.pl` generates and the kernel table entry in `syscall.c`.

```
dev1@dev1:~/xv6-riscv$ cd ~/xv6-riscv
dev1@dev1:~/xv6-riscv$ grep -n "pause\|sleep" user/user.h
25:int pause(int);
```

Figure 1.1 – The grep result in user.h

1.2 The updated sleep.c

The program checks that exactly one argument is given; otherwise it prints the usage line on stderr (fd 2) and exits with status 1. It then converts the argument with `atoi()` and calls `pause()`. `$U/_sleep\` and `$U/_args\` are in `UPROGS`; `cat -A` shows that each line starts with a real TAB (`^I`) and ends with the backslash, with nothing after it (Figure 1.2).

```
dev1@dev1:~/xv6-riscv$ cat user/sleep.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  if(argc != 2){
    fprintf(2, "usage: sleep <ticks>\n");
    exit(1);
  }
  pause(atoi(argv[1]));   // was sleep() before Aug 2025
  exit(0);
}
dev1@dev1:~/xv6-riscv$ grep -n '_sleep\|_args' Makefile | cat -A | sed 's/\^I/<TAB>/'
154:<TAB>$U/_sleep\$
155:<TAB>$U/_args\$
```

Figure 1.2 – The updated sleep.c and its UPROGS line

1.3 Running sleep in xv6

After `make qemu`, `sleep 20` returned to the prompt after 1.97 s of wall time. xv6 programs the timer interrupt every 1 000 000 cycles of the 10 MHz QEMU clock, i.e. one tick every 0.1 s, so 20 ticks ≈ 2 s, as expected. Without an argument the program prints `usage: sleep <ticks>` (Figure 1.3).

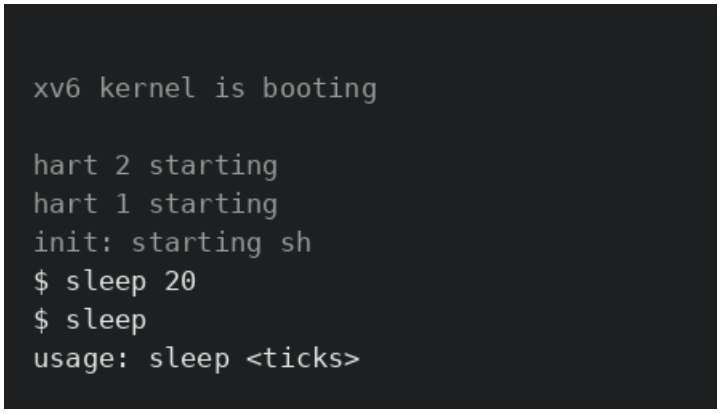


Figure 1.3 – sleep running in xv6, with and without an argument

2 BASH VERSUS THE XV6 SHELL

2.1 Comparison table

In Bash every command ran in `~/shtest`, which contains only `README`; in xv6 it ran at the `$` prompt in `/` (Figures 2.1 and 2.2).

Command	Tests	bash	xv6	Why
`echo hello > f ; cat < f`	redirection	`hello`	`hello`	Both shells open the file and put it on fd 1 / fd 0 before `exec`.
`ls | grep READ`	pipe	`README`	`README 2 2 2441`	Both create a `pipe()`. xv6 `ls` always prints type, inode and size, so `grep` matches the whole line.
`(echo a; echo b) | wc`	grouping, list	`2 2 4`	`2 2 4`	Both parse `( )` as a block and run the list in a subshell.
`echo one ; echo two`	sequence	`one` / `two`	`one` / `two`	`;` builds a LIST node: run left, wait, run right.
`echo bg &`	background	`bg`, and the prompt returns at once	`bg`, printed after the next `$`	BACK node: the shell does not `wait()`, so the prompt can appear before the job's output.
`ls *`	globbing	`README f`	`ls: cannot open *`	Only Bash expands `*`; xv6 passes the literal `*` to `ls`.
`echo $HOME`	variables	`/home/dev1`	`$HOME`	xv6 has no variables and no environment.
`echo "a b"`	quotes	`a b`	`"a b"`	xv6 does not treat quotes specially: they stay in the arguments.
`mkdir d ; cd d ; ls ..`	PATH	`README d f`	`exec cd failed`, then the listing of `/`	In xv6 `cd` is a built-in only at the start of a line; inside a list it is sent to `exec` and fails. `ls ..` then runs in `/`, whose parent is `/` itself.
arrow up, Tab, Ctrl+R	history, completion	recall, completion, reverse search	`exec failed` for each	The xv6 console passes the keys as ordinary characters (`ESC [ A`, TAB, `^R`); sh tries to execute them.

Table 2.1 – The same commands in bash and in the xv6 shell

```
dev1@dev1:~/xv6-riscv$ mkdir ~/shtest && cd ~/shtest && touch README
dev1@dev1:~/shtest$ echo hello > f ; cat < f
hello
dev1@dev1:~/shtest$ ls | grep READ
README
dev1@dev1:~/shtest$ (echo a; echo b) | wc
      2       2       4
dev1@dev1:~/shtest$ echo one ; echo two
one
two
dev1@dev1:~/shtest$ echo bg &
bg
dev1@dev1:~/shtest$ ls *
README
f
dev1@dev1:~/shtest$ echo $HOME
/home/dev1
dev1@dev1:~/shtest$ echo "a b"
a b
dev1@dev1:~/shtest$ mkdir d ; cd d ; ls ..
README
d
f
dev1@dev1:~/shtest/d$ <Up>  ->  mkdir d ; cd d ; ls ..   (previous line recalled)
dev1@dev1:~/shtest/d$ cat ../RE<Tab>  ->  cat ../README   (file name completed)
dev1@dev1:~/shtest/d$ <Ctrl+R> grep  ->  (reverse-i-search)`grep': ls | grep READ
```

Figure 2.1 – The commands run in bash

```
$ echo hello > f ; cat < f
hello
$ ls | grep READ
README          2 2 2441
$ (echo a; echo b) | wc
2 2 4
$ echo one ; echo two
one
two
$ echo bg &
bg
$ ls *
ls: cannot open *
$ echo $HOME
$HOME
$ echo "a b"
"a b"
$ mkdir d ; cd d ; ls ..
exec cd failed
.               1 1 1024
..              1 1 1024
README          2 2 2441
cat             2 3 35344
echo            2 4 34256
forktest        2 5 17344
grep            2 6 42696
init            2 7 34728
kill            2 8 34192
ln              2 9 34008
ls              2 10 41480
mkdir           2 11 34232
rm              2 12 34224
sh              2 13 56104
stressfs        2 14 35104
usertests       2 15 200216
grind           2 16 50248
wc              2 17 36224
zombie          2 18 33600
logstress       2 19 36144
forphan         2 20 35048
dorphan         2 21 34512
sync            2 22 33664
sleep           2 23 33984
args            2 24 33976
console         3 25 0
f               2 26 6
d               1 27 32
$ <Up>
exec  failed
$ ec<Tab>
exec ec failed
$ <Ctrl+R>grep
^Rgrep
exec ^Rgrep failed
```

Figure 2.2 – The same commands run in xv6

2.2 ls after cd d

```
$ cd d
$ ls
exec ls failed
$ /ls
.              1 27 32
..             1 1 1024
$ ../ls
.              1 27 32
..             1 1 1024
$ /echo still works from d
still works from d
$ cd /
```

Figure 2.3 – ls failing after cd d, and the working alternatives

2.3 Answers to the Observe questions

a) **Features both shells have, and the ones only bash has.** Both: running a program with arguments, < and > redirection (xv6 also has >>), pipes `|`, sequences `;`, background `&`, grouping `( )`, and the built-in `cd`. Only Bash: globbing (`* ? [ ]`), variables and the environment (`$HOME`), quoting (`" "`, `' '`, \), PATH search, history, Tab completion, Ctrl+R, `&&` and `||`, job control (`jobs`, `fg`, `kill %1`), command substitution, and scripting (`if`, `for`, functions).

b) **Why does ls stop working after cd d, and how can it still be run?** xv6 has no PATH: `exec("ls", ...)` opens `ls` relative to the current directory. All xv6 programs live in the root directory `/`, where `mkfs` put them, so the bare name works only while the current directory is `/`. Inside `d` it gives `exec ls failed`. A path to the program works: `/ls` (absolute) or `../ls` (relative). Both list `d`, and `/echo` works the same way (Figure 2.3).

c) **What would I miss most, and what is easiest to add?** I would miss globbing with quoting the most, together with history and Tab completion, because every file name would have to be typed in full. The easiest to add is a PATH fallback: when `exec(argv[0])` fails and the name has no `/`, `runcmd()` can retry `exec` with `"/"` prepended. That is about five lines in `case EXEC` and needs no new data structures.

3 WHO EXPANDS *? A PROGRAM THAT PRINTS ITS ARGV

3.1 args.c and args_linux.c

```
dev1@dev1:~/shtest/d$ cd ~/xv6-riscv
dev1@dev1:~/xv6-riscv$ cat user/args.c
#include "kernel/types.h"
#include "user/user.h"

int
main(int argc, char *argv[])
{
  printf("argc = %d\n", argc);
  for(int i = 0; i < argc; i++)
    printf("argv[%d] = [%s]\n", i, argv[i]);
  exit(0);
}
dev1@dev1:~/xv6-riscv$ grep -n '_args' Makefile
155:□$U/_args\
```

Figure 3.1 – args.c and the UPROGS line

`args_linux.c` is the same program with `#include <stdio.h>` and `return 0`. It was compiled with `gcc -Wall -o args args_linux.c` without warnings and run in `~/argtest`, which contains `a.c`, `b.c` and `args_linux.c`.

```
$ args one   two   three
argc = 4
argv[0] = [args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
$ args *.c "x y"
argc = 4
argv[0] = [args]
argv[1] = [*.c]
argv[2] = ["x]
argv[3] = [y"]
$ args $HOME
argc = 2
argv[0] = [args]
argv[1] = [$HOME]
$ args 'single quoted'
argc = 3
argv[0] = [args]
argv[1] = ['single]
argv[2] = [quoted']
```

Figure 3.2 – args output in xv6

```
dev1@dev1:~/xv6-riscv$ cd ~/argtest
dev1@dev1:~/argtest$ gcc -Wall -o args args_linux.c && ls
a.c
args
args_linux.c
b.c
dev1@dev1:~/argtest$ ./args one   two   three
argc = 4
argv[0] = [./args]
argv[1] = [one]
argv[2] = [two]
argv[3] = [three]
dev1@dev1:~/argtest$ ./args *.c "x y"
argc = 5
argv[0] = [./args]
argv[1] = [a.c]
argv[2] = [args_linux.c]
argv[3] = [b.c]
argv[4] = [x y]
dev1@dev1:~/argtest$ ./args $HOME
argc = 2
argv[0] = [./args]
argv[1] = [/home/dev1]
dev1@dev1:~/argtest$ ./args 'single quoted'
argc = 2
argv[0] = [./args]
argv[1] = [single quoted]
```

Figure 3.3 – args_linux output in bash

3.2 Answers to the Observe questions

a) **Both outputs for args *.c ”x y”, and how many arguments each program received.**

bash – `argc = 5`

`argv[0] = [./args]`

`argv[1] = [a.c]`

`argv[2] = [args_linux.c]`

`argv[3] = [b.c]`

`argv[4] = [x y]`

xv6 – `argc = 4`

`argv[0] = [args]`

`argv[1] = [*.c]`

`argv[2] = ["x]`

`argv[3] = [y"]`

The Linux program received 4 arguments after its name: `*.c` became three file names, and `"x y"` became one argument without the quotes. The xv6 program received 3: the literal `*.c`, and `"x y"` cut at the space into `"x` and `y"`, with the quote characters still present.

b) **Who turns *.c into file names and removes the quotes?** The shell. It is the same `exec` system call on both systems, and the program code is the same. Only the shell differs, and the results

differ with it. The kernel copies the strings it is given onto the new process's stack, and the program prints what it receives. Bash expands and removes quotes before `execve()`; the xv6 shell does neither. The other two lines confirm this: `args $HOME` gives `/home/dev1` in Bash and the literal `$HOME` in xv6, and `'single quoted'` is one argument in Bash but two (`'single` and `quoted'`) in xv6.

c) **What happened to the extra spaces in one two three?** Both shells used them only as separators: `argc = 4` with `[one] [two] [three]`. The xv6 tokenizer `gettoken()` skips any run of characters from `whitespace[] = " \t\r\n\v"`, and Bash word splitting does the same. To keep the spaces in Bash the argument must be quoted (`"one two"`); in xv6 they cannot be kept at all.

```
dev1@dev1:~/argtest$ cd ~/xv6-riscv
dev1@dev1:~/xv6-riscv$ wc -l user/sh.c
499 user/sh.c
dev1@dev1:~/xv6-riscv$ grep -n "#define" user/sh.c
8:#define EXEC  1
9:#define REDIR 2
10:#define PIPE  3
11:#define LIST  4
12:#define BACK  5
14:#define MAXARGS 10
```

Figure 4.1 – The command kinds defined in sh.c

```
dev1@dev1:~/xv6-riscv$ grep -n "case " user/sh.c
75:  case EXEC:
83:  case REDIR:
93:  case LIST:
101:  case PIPE:
125:  case BACK:
284:  case 0:
286:  case '|':
287:  case '(':
288:  case ')':
289:  case ';':
290:  case '&':
291:  case '<':
294:  case '>':
391:    case '<':
394:    case '>':
397:    case '+': // >>
469:  case EXEC:
475:  case REDIR:
481:  case PIPE:
487:  case LIST:
493:  case BACK:
```

Figure 4.2 – Where each kind is run

```
dev1@dev1:~/xv6-riscv$ grep -n "cd" user/sh.c | head
170:        fprintf(2, "cannot cd %s\n", cmd + 3);
dev1@dev1:~/xv6-riscv$ sed -n '158,175p' user/sh.c

  // Read and run input commands.
  while (getcmd(buf, sizeof(buf)) >= 0) {
    char *cmd = buf;
    while (*cmd == ' ' || *cmd == '\t')
      cmd++;
    if (*cmd == '\n') // is a blank command
      continue;
    if (cmd[0] == 'c' && cmd[1] == 'd' && cmd[2] == ' ') {
      // Chdir must be called by the parent, not the child.
      cmd[strlen(cmd) - 1] = 0; // chop \n
      if (chdir(cmd + 3) < 0)
        fprintf(2, "cannot cd %s\n", cmd + 3);
    } else {
      if (fork1() == 0)
        runcmd(parsecmd(cmd));
      wait(0);
    }
```

Figure 4.3 – The code that handles cd

```
dev1@dev1:~/xv6-riscv$ grep -n "'\*'" user/sh.c ; echo "exit: $?"
exit: 1
dev1@dev1:~/xv6-riscv$ grep -n '\$' user/sh.c
137:  write(2, "$ ", 2);
dev1@dev1:~/xv6-riscv$ grep -n 'glob\|getenv\|HOME\|PATH' user/sh.c ; echo "exit: $?"
exit: 1
dev1@dev1:~/xv6-riscv$ grep -n 'whitespace\|symbols' user/sh.c | head -3
268:char whitespace[] = " \t\r\n\v";
269:char symbols[] = "<|>&;()";
278:  while (s < es && strchr(whitespace, *s))
```

Figure 4.4 – Searching sh.c for code that handles * or $

4.1 How sh.c runs a line

`main()` reads a line with `getcmd()`. If the line is not `cd`, it calls `fork1()`, and the child runs `runcmd(parsecmd(buf))` while the parent waits. `parsecmd()` is a recursive-descent parser (`parseline`, `parsepipe`, `parseexec`, `parseredirs`, `parseblock`) that builds a tree of command structures. The switch at lines 75–125 executes each node; the second switch, at lines 469–493, in `nulterminate()`, only puts `'\0'` after each word. The character switch at lines 284–297 in `gettoken()` defines the only special characters: `|` `(` `)` `;` `&` `<` `>` and `>>` (returned as `'+'`).

4.2 Answers to the Observe questions

a) **The five kinds of command and the character that produces each** (lines 8–12, Figure 4.1): `EXEC` (1) – a plain command with no operator, run with `exec()`; `REDIR` (2) – `<`, `>` and `>>`: closes fd 0 or 1 and opens the file, which then gets the lowest free fd; `PIPE` (3) – `|`: `pipe()`, two `fork1()` calls, each child replaces fd 1 or fd 0 with an end of the pipe; `LIST` (4) – `;`: fork and wait for the left side, then run the right side; `BACK` (5) – `&`: fork and do not wait. Parentheses do not form a sixth kind: `parseblock()` only groups other nodes.

b) **Where is cd handled, and why is it not a separate program?** In `main()`, lines 166–171 (Figure 4.3): if the line starts with `cd` followed by a space, the parent shell itself calls `chdir(cmd + 3)` and never forks. The comment in the source says "Chdir must be called by the parent, not the child." The current directory belongs to each process: a child's `chdir()` would end with the child and leave the shell unchanged. This also explains `exec cd failed` in Part 2: inside a list the line does not start with `cd`, so `cd` is sent to `exec`, and there is no program with that name.

c) **Code that handles * or $, and what that explains about Part 2.** There is none (Figure 4.4). `grep` finds no `'*'` character constant. The only `$` in the file is the prompt string `write(2, "$ ", 2)`, and there is no `getenv`, `HOME`, `PATH` or glob code. The special characters are only `symbols[] = "<|>&;()"`. So `*`, `$` and `"` are ordinary word characters for xv6, which is exactly why `ls *`, `echo $HOME`, `echo "a b"` and `args *.c "x y"` behaved literally in Parts 2 and 3.

CONCLUSIONS

A complete shell fits in 499 lines because the core is small: read a line, parse it into a tree of five node kinds, then `fork`, rearrange file descriptors and `exec`. Redirection, pipes, sequences, background jobs and grouping all worked the same way in xv6 as in Bash, because they are built only from `fork`, `exec`, `pipe`, `dup`, `close` and `wait`. The only built-in is `cd`, because it has to change the state of the shell process itself.

Everything else that Bash adds is user convenience implemented in the shell, not in the kernel: globbing, variables, quoting, PATH search, history and completion. The `args` experiment showed this directly. With the same system call and the same program, only the shell changed, and `argc` went from 4 to 5 and `"x y"` from two broken words to one clean argument. The xv6 failures (`ls: cannot open *`, `exec ls failed` after `cd d`, `exec cd failed` inside a list) each match a specific missing feature in `sh.c`.

Part 1 showed that a system call is a contract with three parts: a declaration in `user.h`, a stub generated from `usys.pl`, and a kernel table entry. Renaming `sleep` to `pause` broke old programs at link time, not at compile time. After the fix, `sleep 20` took 1.97 s, which matches the 0.1 s xv6 timer tick.

REFERENCES

1. Cox, R., Kaashoek, M. F., & Morris, R. *xv6: a simple, Unix-like teaching operating system* (RISC-V edition). MIT, 2024. Chapter 1: Operating system interfaces.

2. `mit-pdos/xv6-riscv`, `user/sh.c`, `user/usys.pl`, `kernel/start.c`. https://github.com/mit-pdos/xv6-riscv

3. Free Software Foundation. *Bash Reference Manual*, version 5.2.